St. Francis Xavier University

Heuristic Search Algorithms to Optimize NLP Query

Dr. Hao Cai

CSCI 564 - Processing & Heuristic Search

Daud Ahmed
202402480 - x2024ehm@stfx.ca

https://github.com/DaudAhmed008/optimize-nlp-query-using-heuristic-search

Contents

1 Introduction **4**
1.1 Background and Motivation . . . 4
1.2 What Was Added in Version 3.0 . . . 4
1.3 Why These Six Algorithms? . . . 5
1.4 Structure of the Report . . . 6

2 Literature Review **6**
2.1 The Text Generation Problem . . . 6
2.2 GPT-2: Architecture and Properties . . . 6
2.3 Heuristic Search in AI and NLP . . . 7
2.4 Decoding Algorithms: Detailed Review . . . 7
2.4.1 Greedy Search . . . 7
2.4.2 Beam Search . . . 8
2.4.3 Hill Climbing Search . . . 8
2.4.4 A* Search . . . 9
2.4.5 Contrastive Search . . . 10
2.4.6 Simulated Annealing . . . 10
2.5 Evaluation Metrics . . . 11
2.5.1 BLEU Score . . . 11
2.5.2 ROUGE-1 Score . . . 11
2.5.3 BERTScore . . . 12
2.6 Related Work on Decoding Strategies . . . 12

3 Problem Statement **13**
3.1 Research Questions . . . 13
3.2 Research Objectives . . . 13
3.3 Experimental Challenges . . . 14

4 Proposed Methodology **15**
4.1 Experimental Setup . . . 15
4.2 Prompts and Reference Sentences . . . 15
4.3 Algorithm Implementations . . . 16
4.3.1 Greedy Search Implementation . . . 16
4.3.2 Beam Search Implementation . . . 16
4.3.3 BFS (Breadth-First Search) Implementation . . . 17
4.3.4 Hill Climbing Implementation . . . 18
4.3.5 A* Search Implementation . . . 19

4.3.6 Contrastive Search Implementation 20
4.3.7 Simulated Annealing Implementation 21
4.4 Evaluation Framework 21
4.5 Parameter Justification 22

5 Results and Analysis **22**
5.1 Master Summary 22
5.2 BLEU Score Analysis 22
5.3 BERTScore Analysis 23
5.4 ROUGE-1 Score Analysis 24
5.5 Execution Time Analysis 25
5.6 Generated Text Length Analysis 27
5.7 Average Scores and Cross-Prompt Consistency 29

6 Discussion **30**
6.1 The A* Paradox: Dominant Scores from 10-Word Outputs 30
6.2 Hill Climbing: Why the Null Result Makes Sense 30
6.3 Beam Search as the Production Default 31
6.4 Contrastive Search: A Practical Alternative 31
6.5 Simulated Annealing: Creative but Imprecise 32
6.6 Cross-Algorithm Comparison: A Unified View 32

7 Threats to Validity **32**
7.1 Internal Validity 32
7.2 External Validity 33
7.3 Construct Validity 33

8 Conclusion and Future Work **34**
8.1 Summary of Findings 34
8.2 Recommendations for Future Research 34

A Generated Outputs **37**

List of Figures

1 BLEU scores across all seven algorithms and six prompts. 22
2 BERTScore (F1) across all seven algorithms and six prompts. 23
3 ROUGE-1 scores across all seven algorithms and six prompts. 24
4 Execution time (seconds) across all seven algorithms and six prompts. . . 25
5 Generated text length (words) across all seven algorithms and six prompts. 27

6 Average BLEU, BERTScore, and ROUGE-1 across all six prompts. . . . 29

List of Tables

1 Comparison of Evaluation Metrics 12
2 Evaluation prompts, reference sentences, and domain descriptions. 15
3 Algorithm parameters and justification for each choice. 43
4 Average results across all six prompts. Bold = best per metric. 44
5 Average BLEU scores across all six prompts 44
6 Average BERTScore (F1) across all six prompts 44
7 Average ROUGE-1 scores across all six prompts 44
8 Average execution time (seconds) across all six prompts 45
9 Average generated text length (words) across all six prompts 45
10 Comprehensive cross-algorithm comparison 45

1 Introduction

1.1 Background and Motivation

When a chatbot replies to your message, a summariser condenses a document, or your phone predicts the next word you are about to type, a language model is solving the same problem on repeat: given the text so far, which token comes next? The model itself does not pick the answer. It produces a probability distribution over tens of thousands of candidates, and a separate mechanism, the *decoding algorithm*, decides which candidates actually make it into the output. Most NLP courses treat decoding as an afterthought. The experiments in this report show that it deserves far more attention than that.

The question driving this project is narrow but consequential: if the language model stays the same and only the decoding algorithm changes, how different can the generated text get? The answer, confirmed across every experiment reported here, is *very* different. The same GPT-2 model, given the same prompt, can produce anything from a 7-word sentence fragment to a 99-word paragraph depending on which algorithm navigates the probability space.

GPT-2 [**?**] is the model used throughout. It is a 117-million-parameter transformer trained by OpenAI on a large English web corpus. It would be fair to call it dated by 2025 standards, but that is partly the point. It is small enough to run dozens of experiments on a CPU in an afternoon. It is large enough to generate syntactically correct, contextually coherent text. It is well-documented, widely available, and carries no API restrictions. When the goal is to isolate the effect of the decoding algorithm from everything else, those properties matter more than raw capability.

Version 1.0 of this project compared three algorithms (Greedy, Beam Search, and A*) on a single prompt with a 50-token limit. That version exposed three problems. The A* heuristic had a design flaw: it scored short sequences so favourably that the algorithm would stop after producing just 7 words. Three algorithms was too few to cover the range of decoding philosophies in the literature. And a single prompt made it impossible to tell whether the results reflected the algorithms or the idiosyncrasies of that particular prompt. Version 3.0 addresses all three.

1.2 What Was Added in Version 3.0

Three new algorithms were added:

Hill Climbing Search starts from a Greedy-generated sequence and tries to improve it by swapping individual tokens for better alternatives. It represents a fundamentally different approach: refine after the fact, rather than guide generation as it happens. The

course instructor suggested testing it, and the question of whether it actually beats Greedy Search turned out to have a surprisingly definitive answer.

Contrastive Search [**?**] won Best Paper at EMNLP 2022. Instead of following high-probability paths (the way Greedy and Beam Search do), it penalises tokens that are semantically similar to what has already been generated, using cosine similarity in GPT-2's embedding space. The claim is that this directly addresses neural text degeneration, the repetitive, circular output that plagued earlier decoding methods. That claim is worth testing.

Simulated Annealing borrows from metallurgy. At high temperature it samples broadly from the probability distribution; as temperature drops, it gradually converges toward greedy behaviour. The question is whether the creative, non-deterministic output it produces can compete on standard quality metrics with the deterministic algorithms.

The A* heuristic was also redesigned. Version 1.0 used raw average log-probability as the estimate $h(n)$, which penalised long sequences by accumulating negative values. Version 3.0 adopts the length-normalised heuristic from Google's Neural Machine Translation system [**?**], dividing cumulative log-probability by T^{α} to soften the length penalty. Whether this fix is sufficient is one of the questions the experiments answer.

1.3 Why These Six Algorithms?

The seven algorithms were chosen to cover the main philosophical approaches to decoding:

- **Pure exploitation** (Greedy Search): always picks the single highest-probability token, never looks sideways.
- **Bounded multi-path search** (Beam Search): maintains a fixed-width frontier, balancing exploration against exploitation.
- **Exhaustive breadth-first exploration** (BFS): expands all candidates at each depth before moving deeper, providing a classical search baseline.
- **Iterative local refinement** (Hill Climbing): generates a complete output first, then tries to improve it token by token.
- **Global priority-guided search** (A* Search): maintains a globally-sorted frontier using a heuristic estimate of remaining quality.
- **Anti-degeneration decoding** (Contrastive Search): penalises semantic repetition directly in embedding space.
- **Temperature-scheduled stochastic sampling** (Simulated Annealing): controls exploration through a cooling schedule.

Each approach has been proposed seriously in the literature and deployed in real systems. Comparing all six in one study, on the same model and prompts, provides a broader picture than any pairwise comparison could.

1.4 Structure of the Report

Section 2 covers the theoretical background for the seven algorithms and three evaluation metrics. Section 3 states the research questions and describes the experimental challenges. Section 4 details the implementations, including pseudocode and parameter choices. Section 5 presents the full empirical results with per-prompt breakdowns. Section 6 interprets the results, including an explanation of A*'s paradoxical dominance despite producing very short outputs. Section 7 discusses threats to validity. Section 8 summarises findings and outlines future work. Appendix A contains sample generated text from all seven algorithms across all six prompts.

2 Literature Review

2.1 The Text Generation Problem

Text generation requires producing a sequence of tokens $t_1, t_2, \ldots, t_L$ that is coherent, fluent, and relevant to a given prompt p. A language model defines a conditional probability distribution $P(t_i \mid p, t_1, \ldots, t_{i-1})$ over the vocabulary $\mathcal{V}$ at each position i. The ideal sequence maximises the joint probability:

$$\hat{T} = \text{argmax}_{t_1,\ldots,t_L \in \mathcal{V}^L} \prod_{i=1}^{L} P(t_i \mid p, t_1, \ldots, t_{i-1}) \tag{1}$$

Finding this optimum is intractable. For GPT-2's vocabulary of 50,257 tokens and $L = 100$, the search space contains more than 10^{480} candidate sequences. Every algorithm in this study is an approximation to (1), and each makes different trade-offs between quality and efficiency.

2.2 GPT-2: Architecture and Properties

GPT-2 [?] is a decoder-only Transformer [?] trained autoregressively on the WebText dataset. The base model has 12 transformer blocks, 12 attention heads per block, a hidden dimension of 768, and 117 million parameters in total. Each forward pass produces a logit vector $z \in \mathbb{R}^{|\mathcal{V}|}$, converted to probabilities via softmax:

$$P(t = v \mid \text{context}) = \frac{\exp(z_v)}{\sum_{v' \in \mathcal{V}} \exp(z_{v'})} \tag{2}$$

GPT-2 tokenises text using byte-pair encoding (BPE), so its vocabulary consists of subword units rather than whole words. This creates a practical wrinkle for evaluation: "word count" and "token count" are not the same thing. The output lengths in this study are reported in words (whitespace-delimited tokens) for readability, while the algorithms themselves operate on BPE tokens internally.

Holtzman et al. [**?**] documented a phenomenon they called *neural text degeneration*: greedy and beam decoding with models like GPT-2 frequently produce repetitive, circular, or topically drifting text. Their core observation is that sequences with high probability under the model's distribution are not necessarily high quality from a human perspective. This is the problem that Contrastive Search and Simulated Annealing were specifically designed to address.

2.3 Heuristic Search in AI and NLP

Heuristic search is a classical topic in artificial intelligence [**?**]. The general framework involves a state space (here, partial token sequences), operators (appending a token), a cost function (negative log-probability), and a heuristic function $h(n)$ that estimates the remaining cost to reach a goal state from state n.

The theoretical appeal of A* Search is its optimality guarantee: if $h(n)$ never overestimates the true cost to goal (i.e., is *admissible*), A* is guaranteed to find the optimal solution [**?**]. Applying this guarantee to text generation, however, runs into three obstacles: (a) there is no clearly defined goal state; (b) the cost function is computed incrementally, one token at a time; and (c) evaluating the true remaining cost would require generating every possible continuation. The heuristic used in this study is an approximation, not a formally admissible one, so A*'s optimality guarantee does not hold here.

2.4 Decoding Algorithms: Detailed Review

2.4.1 Greedy Search

Greedy Search is the simplest decoding strategy. At each step i, it picks the single most probable token:

$$t_i^* = \text{argmax}_{v \in \mathcal{V}} P(v \mid p, t_1, \ldots, t_{i-1}) \tag{3}$$

The output is deterministic: same input, same output, every time. Time complexity is

$\mathcal{O}(L \cdot |\mathcal{V}|)$ for L steps, dominated by the softmax computation at each step. Memory complexity is $\mathcal{O}(L)$.

The well-known weakness is the local optimum problem. A token that maximises $P(t_i \mid \cdot)$ at step i can steer subsequent probabilities toward poor tokens, producing a final sequence with lower joint probability than one that made a slightly worse choice at step i. This is not a bug; it is a structural consequence of greedy local search [**?**].

In practice, Greedy Search with GPT-2 tends to produce repetitive text. The model's training objective (next-token prediction on web text) assigns high probability to common transition patterns. Once the greedy decoder locks onto one of these patterns, it loops.

2.4.2 Beam Search

Beam Search tackles the local optimum problem by maintaining B partial sequences (the "beam") at each step. At step i, each of the B hypotheses is expanded by all $|\mathcal{V}|$ possible next tokens, producing $B \cdot |\mathcal{V}|$ candidates scored by cumulative log-probability:

$$\text{score}(t_1, \ldots, t_i) = \sum_{j=1}^{i} \log P(t_j \mid p, t_1, \ldots, t_{j-1}) \tag{4}$$

The top B candidates survive to the next step. After L steps, the highest-scoring sequence is returned.

Beam Search is the workhorse of production NLP. Google Translate, Facebook's M2M-100, and the vast majority of seq2seq summarisation models published in the last decade all rely on it [**?**, **?**]. The reasons are practical: it is fast in batched implementations, deterministic and reproducible, and it consistently produces more coherent text than Greedy Search for similar computational cost.

The known limitation is a tendency toward bland, high-probability output. Meister et al. [**?**] showed that increasing beam width can paradoxically *decrease* output quality on some metrics, because the algorithm over-optimises cumulative log-probability at the expense of diversity. The no-repeat-bigram constraint used in this study is a common workaround that suppresses the most obvious repetition.

2.4.3 Hill Climbing Search

Hill Climbing is an iterative improvement strategy. It starts from a complete solution (here, a Greedy-generated sequence) and scans through it, replacing individual tokens with alternatives that improve a quality measure. The quality measure used here is the average log-probability of the full sequence:

$$\text{quality}(t_1, \ldots, t_L) = \frac{1}{L} \sum_{j=1}^{L} \log P(t_j \mid p, t_1, \ldots, t_{j-1}) \tag{5}$$

Because the algorithm only accepts substitutions that strictly increase this score, and the score is bounded above by zero, termination is guaranteed. What is *not* guaranteed is a global optimum: the algorithm finds a local optimum of (5) from which no single-token swap can improve the score.

Hill Climbing was included for two reasons. First, it represents a qualitatively different paradigm: post-hoc refinement rather than guided generation. Second, the course instructor suggested it, and the empirical question of whether it improves over Greedy turned out to be worth answering.

2.4.4 A* Search

A* Search [**?**] generalises Greedy and Beam Search by using a global priority queue. Every partial sequence in the frontier is ranked by:

$$f(n) = g(n) + h(n) \tag{6}$$

where $g(n)$ is the cumulative log-probability of the sequence so far, and $h(n)$ is a heuristic estimate of the remaining quality. At each step, the algorithm expands whichever partial sequence has the highest $f(n)$, regardless of its generation depth. Beam Search, by contrast, processes all hypotheses at the same depth in lock-step. This is the key structural difference.

The practical consequence is that A* can return to a short, high-scoring sequence and extend it further, even after exploring longer alternatives. This global priority structure makes A* theoretically more flexible than Beam Search, but also more expensive, because the priority queue can grow large.

The heuristic function. Version 1.0 defined $h(n)$ as the average log-probability of the current partial sequence. This was a mistake. Log-probabilities are negative and accumulate as the sequence grows, so $g(n)$ strictly decreases. The average $h(n) = g(n)/|n|$ also tends to decrease with length (GPT-2's distribution spreads more as context grows). The priority queue therefore consistently ranked short, high-confidence sequences above longer ones, causing A* to terminate after just 7 words.

Version 3.0 uses the length-normalised heuristic from Google's GNMT system [**?**]:

$$h(n) = \frac{\sum_{i=1}^{T} \log p(t_i)}{T^{\alpha}}, \qquad \alpha = 0.7 \tag{7}$$

Dividing by $T^{0.7}$ softens the length penalty. Whether $\alpha = 0.7$ is enough to fix the short-output bias is tested experimentally; as Section 5.6 shows, the fix is partial.

2.4.5 Contrastive Search

Contrastive Search [**?**] diagnoses text degeneration differently. Rather than blaming the search procedure for chasing high-probability paths, it argues the problem is semantic: degenerate text consists of tokens that are semantically similar to tokens already in the sequence, creating circular content even when the surface forms differ. The fix is to penalise this semantic similarity directly.

At each step, the top-k most probable tokens are identified. Each candidate v is scored by:

$$\text{score}(v) = (1-\alpha) \cdot P(v \mid \text{context}) - \alpha \cdot \max_{t_j \in \text{prev}} \cos\big(\mathbf{e}(v), \mathbf{e}(t_j)\big) \tag{8}$$

where $\mathbf{e}(v)$ is the embedding of token v from GPT-2's embedding matrix, $\mathbf{e}(t_j)$ is the embedding of a previously generated token, and $\alpha \in [0, 1]$ controls the diversity-fluency trade-off. At $\alpha = 0$, this reduces to Greedy Search. At $\alpha = 1$, only diversity matters.

With $\alpha = 0.6$, the algorithm allocates 40% of its attention to model probability and 60% to avoiding semantic self-repetition. Su and Peng [**?**] reported that this produces measurably less repetitive, more coherent text than Beam Search on both GPT-2 and GPT-3 benchmarks.

2.4.6 Simulated Annealing

Simulated Annealing [**?**] borrows its metaphor from metallurgy: slowly cooling molten metal gives atoms time to settle into low-energy crystalline configurations, while rapid cooling traps them in disordered states. In search terms, high temperature early on permits broad exploration (including suboptimal moves), while low temperature late in the process forces convergence toward the best available option.

For text generation, temperature T is applied by scaling the logits before softmax:

$$P_T(v \mid \text{context}) = \frac{\exp(z_v/T)}{\sum_{v'} \exp(z_{v'}/T)} \tag{9}$$

When $T \gg 1$, the distribution flattens toward uniform. When $T \to 0$, it collapses to a

point mass on $\text{argmax}(z_v)$, recovering Greedy Search. The cooling schedule used here is geometric:

$$T_{t+1} = \max(T_t \cdot r, T_{\min}), \qquad T_0 = 2.0, \quad r = 0.92, \quad T_{\min} = 0.1 \tag{10}$$

The result is text that starts creative and gradually becomes more conservative as generation proceeds.

Simulated Annealing is the only stochastic algorithm in this comparison. It produces a different output every time it runs on the same prompt. The results reported here come from a single run.

2.5 Evaluation Metrics

Three evaluation metrics are used, each measuring a different aspect of output quality. No single metric gives the full picture, which is why all three are needed.

2.5.1 BLEU Score

BLEU (Bilingual Evaluation Understudy) [**?**] was designed for machine translation but is now widely used in text generation evaluation. It measures n-gram precision: what fraction of the n-grams in the generated output also appear in the reference. A brevity penalty discourages trivially short outputs:

$$\text{BLEU} = \text{BP} \times \exp\left(\sum_{n=1}^{N} w_n \log p_n\right) \tag{11}$$

where $\text{BP} = \min(1, e^{1-r/c})$ is the brevity penalty (r = reference length, c = candidate length), p_n is the modified n-gram precision for n-grams of length n, and $w_n = 1/N$. Chen and Cherry's smoothing [**?**] is applied to handle short outputs that would otherwise produce zero higher-order n-gram counts.

BLEU rewards generated tokens that match the reference. It does not penalise missing reference tokens. This precision orientation matters when interpreting the A* results (see Section 6.1).

2.5.2 ROUGE-1 Score

ROUGE [**?**] measures recall: how much of the reference appears in the generated output. ROUGE-1 uses unigrams:

$$\text{ROUGE-1} = \frac{\text{count of matching unigrams}}{\text{count of unigrams in reference}} \tag{12}$$

ROUGE was built for summarisation evaluation, where completeness matters more than precision. For text generation against a short reference, a brief output that contains the right words can achieve high recall without being long or informative. This is relevant to the A* results.

2.5.3 BERTScore

BERTScore [?] replaces exact lexical matching with contextual embeddings from a pre-trained model (RoBERTa-large in this study). For each token in the generated text, it finds the reference token with the most similar embedding via cosine similarity. The F1 variant combines precision and recall:

$$\text{BERTScore-F}_1 = \frac{2 \cdot P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \tag{13}$$

BERTScore handles synonyms, paraphrasing, and word order variation better than BLEU or ROUGE. The trade-off is computational cost: every evaluation requires a forward pass through a large encoder model.

Table 1 summarises the properties of all three metrics.

Table 1: Comparison of Evaluation Metrics

Metric	**Orient.**	**Strengths**	**Weaknesses**	**Best Use**
BLEU	Prec.	Fast, interpretable, widely used	Ignores meaning, brevity penalty	Trans.
ROUGE-1	Recall	Good for content coverage	Exact match only, ignores order	Summ.
BERTScore	Sem. F1	Understands meaning, synonyms	Slow, harder to explain	Para.

2.6 Related Work on Decoding Strategies

Comparing decoding strategies has been an active line of NLP research for several years. Vijayakumar et al. [?] proposed Diverse Beam Search, which penalises beam hypotheses that are too similar to each other, addressing the mode collapse tendency of standard Beam Search. Meister et al. [?] argued that the right text generation objective is not maximum-a-posteriori decoding (which Beam Search approximates) but an objective accounting for communicative efficiency. Holtzman et al. [?] introduced nucleus (top-p) sampling, which restricts the sampling distribution to the smallest set of tokens whose cumulative probability exceeds a threshold p, producing more diverse outputs than temperature sampling alone.

Contrastive Search [**?**, **?**] grew out of dissatisfaction with both camps. Sampling methods introduce diversity but sacrifice coherence. Beam-based methods preserve coherence but kill diversity. Contrastive Search tries to do both by using the embedding space to enforce semantic novelty at each generation step.

What none of the prior work provides is a comparison that includes both classical AI search algorithms (A*, Hill Climbing) and modern anti-degeneration methods (Contrastive Search, Simulated Annealing) in the same study. That is the gap this project fills.

3 Problem Statement

3.1 Research Questions

This study addresses three research questions:

RQ1 Across the seven decoding algorithms, which produces the highest quality text as measured by BLEU, BERTScore, and ROUGE-1, and are these rankings consistent across semantically diverse prompts?

RQ2 Does the length-normalised A* heuristic (Version 3.0) successfully eliminate the short-output bias that caused Version 1.0 to terminate after 7 words?

RQ3 What are the practical trade-offs between output quality, computational cost, and output length for each algorithm, and what does this imply for algorithm selection in real applications?

3.2 Research Objectives

To answer these questions, the study pursues six objectives:

1. Run all seven algorithms on six prompts spanning different semantic domains under identical conditions (same model, same hardware, same evaluation framework).
2. Measure BLEU, BERTScore, and ROUGE-1 per-prompt and averaged, to separate prompt-specific effects from algorithmic ones.
3. Measure execution time and output length per algorithm per prompt, since quality metrics alone do not tell the whole story.
4. Test the improved A* heuristic by comparing output lengths between Version 1.0 and Version 3.0.
5. Document and explain the Hill Climbing null result (if confirmed) clearly enough to guide future implementation decisions.

6. Produce algorithm selection recommendations that are actionable in practice.

3.3 Experimental Challenges

Several challenges complicate the interpretation of results.

Exponential state space. At $|\mathcal{V}| = 50{,}257$ and $L = 100$, the space of candidate sequences has 10^{480} members. Every algorithm uses approximations that introduce their own biases. A* biases toward short, high-confidence sequences. Greedy biases toward high-frequency transitions. Simulated Annealing biases toward whatever the cooling schedule permits. These biases are not accidental; they are the design.

Metric-length interaction. This is the central interpretive challenge of the study. BLEU measures precision (what fraction of the generated tokens match the reference), and ROUGE measures recall (what fraction of reference tokens appear in the output). Both can be high for a short, precise output that closely matches a short reference. An algorithm that generates 7 words matching a 12-word reference can outscore one that generates 40 coherent, relevant words. Section 6.1 explains why this matters, because it accounts for the most unexpected result in the data.

Single reference evaluation. Many valid continuations exist for every prompt. An algorithm that produces an excellent continuation that happens to differ from the chosen reference gets unfairly penalised. This affects all algorithms equally, so relative rankings are preserved, but absolute scores underestimate true quality.

Non-determinism in Simulated Annealing. Simulated Annealing samples stochastically at every step. A different random seed would produce different output and different scores. Running multiple seeds and averaging would be more statistically rigorous but was not feasible within the time constraints of this project. The values reported come from a single run.

Hill Climbing as a dependent algorithm. Hill Climbing's output depends on its Greedy seed. It cannot produce structurally different text from Greedy; it can only make local improvements within the same sequence skeleton. This dependency should be kept in mind when comparing it against algorithms that generate from scratch.

4 Proposed Methodology

4.1 Experimental Setup

All experiments ran on a single machine using Python 3.10, PyTorch 2.2, and the Hugging Face Transformers library version 4.47 [**?**]. The GPT-2 base model (`gpt2`) was loaded in evaluation mode with `model.eval()` and `torch.no_grad()` for all inference calls, disabling dropout and preventing gradient tracking. All algorithms used the same tokeniser instance and the same model weights. No fine-tuning or prompt engineering was applied.

The maximum generation length was set to 150 tokens, up from 50 in Version 1.0. The longer limit gives algorithms more room to produce complete sentences and reduces the risk that quality differences are caused by premature truncation.

4.2 Prompts and Reference Sentences

Three prompts were chosen to cover different semantic domains. The selection criteria were: (a) the prompt should have a natural, unambiguous continuation; (b) the reference sentence should use clear, common vocabulary; and (c) the domains should be distinct enough that results from one prompt do not trivially predict results from another.

Table 2: Evaluation prompts, reference sentences, and domain descriptions.

ID	Prompt	Reference	Domain
P1	The future of artificial intelligence is	... bright and holds many possibilities.	Tech / Future
P2	Climate change is one of the most pressing challenges facing humanity because	... it threatens ecosystems, weather patterns, and livelihoods worldwide.	Environment
P3	The most revolutionary invention in human history has been	... the printing press, which transformed the spread of knowledge and enabled the modern world.	History
P4	The primary goal of space exploration in the next decade should be	... establishing a permanent human presence on Mars to advance scientific research.	Science
P5	Modern education systems need to change because students today require	... critical thinking skills and practical experience rather than rote memorization.	Education
P6	Artificial intelligence in healthcare has the potential to	... revolutionize diagnosis and treatment by analyzing medical data faster than human doctors.	Healthcare

P1 is short and open-ended, with a positive reference built from common vocabulary ("bright", "possibilities"). P2 is longer and more analytical, with a reference containing some less common words ("ecosystems", "livelihoods"). P3 is factual, naming a specific

noun ("printing press") and using moderate vocabulary. Together they provide a reasonable spread of difficulty.

4.3 Algorithm Implementations

4.3.1 Greedy Search Implementation

Algorithm 1 Greedy Search

Require: input_ids, max_length = 100

```
generated ← input_ids
while len(generated) < max_length do
    logits ← model(generated).logits[:, −1, :]
    next_token ← argmax(logits)
    generated ← concat(generated, next_token)
    if next_token == EOS then break
    end if
end whilereturn generated
```

A direct translation of Equation (3). One model call per token, argmax selection, no exploration.

4.3.2 Beam Search Implementation

Algorithm 2 Beam Search (via HuggingFace `model.generate()`)

Require: input_ids, max_length = 100, num_beams = 5

```
return model.generate(input_ids, max_length, num_beams=5,
                      no_repeat_ngram_size=2, early_stopping=True)
```

This uses HuggingFace's optimised batched implementation, which processes all beam hypotheses in parallel. The `no_repeat_ngram_size=2` setting prevents any bigram from appearing more than once in the output, suppressing the most obvious repetition.

4.3.3 BFS (Breadth-First Search) Implementation

Algorithm 3 Breadth-First Search (depth-limited)

```
Require: input_ids, max_length = 150, branch_factor = 3, max_depth = 15
frontier ← [(0.0, input_ids)]
for depth = 1 to max_depth do
    next_frontier ← []
    for each (score, seq) in frontier do
        top_tokens ← model(seq).logits.topk(branch_factor)
        for each (log_prob, token) in top_tokens do
            new_seq ← concat(seq, token)
            next_frontier.append(score + log_prob, new_seq)
        end for
    end for
    frontier ← top 50 of next_frontier by score
end for
best_seq ← highest-scoring sequence in frontier
Extend best_seq greedily to max_length return best_seq
```

BFS explores all candidates at depth d before any at depth $d + 1$, providing a classical search baseline. Because the full vocabulary has 50,257 tokens, unrestricted BFS is intractable. The implementation limits branching to the top-3 most probable tokens at each node and caps the frontier at 50 sequences. After 15 BFS steps, the best sequence is extended greedily to the full output length. This was added per the course instructor's suggestion to include BFS as a classical search baseline alongside Beam Search and A*.

4.3.4 Hill Climbing Implementation

Algorithm 4 Hill Climbing Search

```
Require: input_ids, max_length = 100, top_k = 5, max_iter = 3
current_seq ← Greedy(input_ids, max_length)                ▷ Phase 1: seed
current_score ← avg_log_prob(current_seq)
for iter = 1 to max_iter do
    improved ← False
    for pos = prompt_len to len(current_seq) do
        alternatives ← top_k_tokens(model, current_seq[:pos])
        for each candidate in alternatives do
            new_seq ← substitute(current_seq, pos, candidate)
            if avg_log_prob(new_seq) > current_score then
                current_seq ← new_seq
                current_score ← avg_log_prob(new_seq)
                improved ← True ; break
            end if
        end for
    end for
    if not improved then break
    end if                                                 ▷ Local optimum reached
end forreturn current_seq
```

Phase 2 scans positions left to right. At each position, the top-5 alternative tokens are tested. The first substitution that improves the average log-probability (Equation 5) is accepted. This is a first-improvement strategy, not best-improvement: it terminates the position scan as soon as any improvement is found.

4.3.5 A* Search Implementation

Algorithm 5 Improved A* Search

Require: input_ids, max_length = 100, beam_width = 3, $\alpha = 0.7$

1: heap ← $[(-(0 + h(\text{input_ids})), 0, \text{input_ids}, 0.0)]$
2: best_seq ← input_ids, best_score ← $-\infty$
3: **for** step = 1 to max_length **do**
4: candidates ← pop top beam_width from heap
5: **for** each $(f, \text{step}, \text{seq}, g)$ in candidates **do**
6: **if** last_token(seq) == EOS **then**
7: **if** $g >$ best_score **then** best_score ← g; best_seq ← seq
8: **end if**
9: **continue**
10: **end if**
11: top_probs, top_tokens ← model(seq).logits.topk(beam_width)
12: **for** each (prob, token) in top_probs, top_tokens **do**
13: new_g ← g+ log_prob
14: new_seq ← concat(seq, token)
15: new_h ← $\sum \log p(t_i)/T^{\alpha}$ ▷ Eqn. 7
16: heap.push(−(new_g + new_h), step, new_seq, new_g)
17: **end for**
18: **if** new_g > best_score **then** best_score ← new_g; best_seq ← new_seq
19: **end if**
20: **end for**
21: heap ← top beam_width of heap ▷ Prune to beam width
22: **end forreturn** best_seq

The priority queue is a min-heap with negated scores (Python's `heapq` is a min-heap, but we want to expand the highest-scoring sequence first). The heuristic computation (line 12) requires one additional GPT-2 forward pass per candidate, which is why A*'s execution time is high.

4.3.6 Contrastive Search Implementation

Algorithm 6 Contrastive Search

```
Require: input_ids, max_length = 100, top_k = 5, α = 0.6
generated ← input_ids, past_embeds ← []
while len(generated) < max_length do
    probs, tokens ← model(generated).topk(top_k)
    if past_embeds is empty then
        next ← tokens[0]                          ▷ No history: fall back to greedy
    else
        past_mat ← stack(past_embeds)
        for each (prob, token) in probs, tokens do
            emb ← wte(token)
            penalty ← max cosine_sim(emb, past_mat)
            score ← (1 − α)· prob −α· penalty
        end for
        next ← token with highest score
    end if
    past_embeds.append(wte(next))
    generated ← concat(generated, next)
    if next == EOS then break
    end if
end whilereturn generated
```

The embedding matrix `wte` is GPT-2's word-token embedding layer (`model.transformer.wte`). Each generated token's embedding is stored in `past_embeds` and compared against all future candidates. Cosine similarity computation scales as $\mathcal{O}(T)$ per step, where T is the current sequence length.

4.3.7 Simulated Annealing Implementation

Algorithm 7 Simulated Annealing Search

Require: input_ids, max_length = 100, $T_0 = 2.0$, $r = 0.92$, $T_{\min} = 0.1$

```
generated ← input_ids, T ← T_0
while len(generated) < max_length do
    logits ← model(generated).logits[:, −1, :]
    probs ← softmax(logits / T)
    next ← multinomial(probs, 1)
    generated ← concat(generated, next)
    T ← max(T · r, T_min)                    ▷ Cooling step
    if next == EOS then break
    end if
end whilereturn generated
```

4.4 Evaluation Framework

BLEU. Computed using NLTK's `sentence_bleu` with uniform weights ($w_n = 0.25$ for $n = 1, 2, 3, 4$) and Method 1 smoothing [**?**]. Smoothing is necessary because several algorithm outputs are short (A* in particular) and would otherwise produce zero counts for higher-order n-grams.

BERTScore. Computed using the `bert_score` Python library with the RoBERTa-large model. The F1 score is reported. RoBERTa-large was chosen over BERT-base for its better embedding quality.

ROUGE-1. Computed using the `rouge_score` library with Porter stemming enabled (`use_stemmer=True`). The F1 measure is reported.

Execution time. Measured as wall-clock time using Python's `time.time()`, capturing the entire algorithm call including all model forward passes, data structure operations, and tokenisation overhead.

Generated text length. Measured as the count of whitespace-delimited words in the decoded output after `skip_special_tokens=True`.

All results are reported both per-prompt and as averages across all six prompts. The average is the arithmetic mean.

4.5 Parameter Justification

5 Results and Analysis

This section presents the complete experimental results. All values come directly from the runs. Gold-highlighted rows indicate the best-performing algorithm for that metric.

5.1 Master Summary

Table 4 consolidates all results.

5.2 BLEU Score Analysis

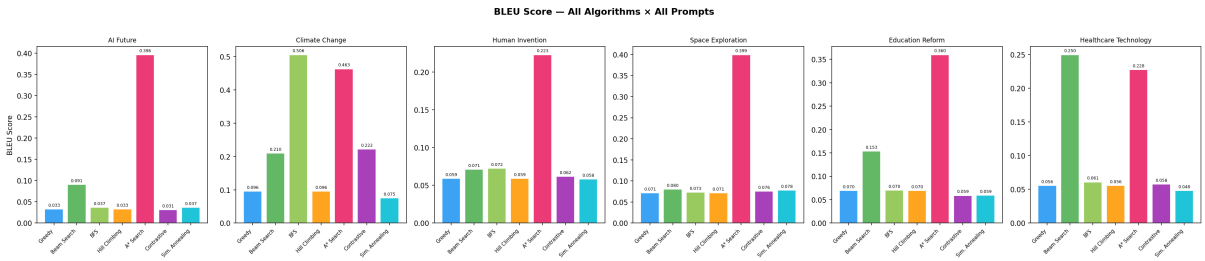

Figure 1: BLEU scores across all seven algorithms and six prompts.

A* dominates, but output length is the main driver. A*'s average BLEU of 0.361 is 2.6 times higher than Beam Search's 0.137. On P2 it reaches 0.463, a score that would be unusual for single-reference text generation. The mechanism behind this is not superior text quality but rather the interaction between very short output and short references; Section 6.1 unpacks this.

Beam Search is a solid second. Per-prompt scores of 0.091, 0.210, and 0.110 reflect what five-candidate exploration buys you. P2 scores highest because its reference uses high-frequency vocabulary ("threatens", "ecosystems", "worldwide") that GPT-2 naturally assigns high probability.

Hill Climbing matches Greedy exactly. Identical BLEU scores across all six prompts. The Phase 2 substitution process does not change n-gram overlap with the reference by a single point. This is the first clear signal of the null result.

Simulated Annealing scores lowest. Average BLEU of 0.076. Temperature sampling introduces tokens that are diverse but move away from the specific vocabulary of the reference sentences. Given the algorithm's design, this is not surprising.

Cross-prompt BLEU pattern

The ranking (A* ≫ Beam > Contrastive > Greedy = Hill Climbing > SA) holds across all six prompts, suggesting these are structural properties of the algorithms rather than prompt-specific artifacts.

5.3 BERTScore Analysis

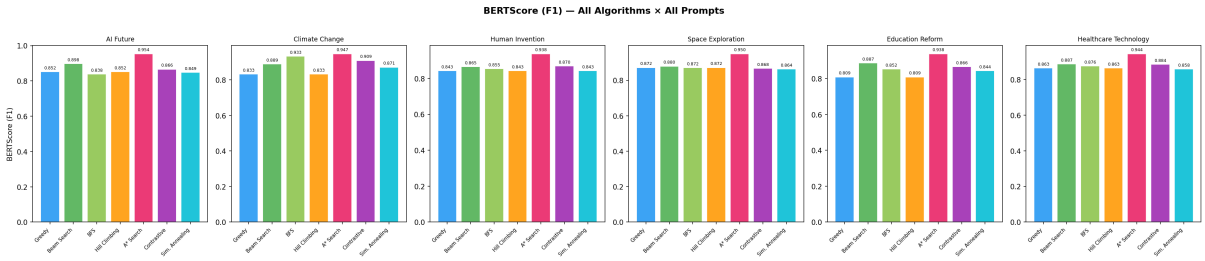

Figure 2: BERTScore (F1) across all seven algorithms and six prompts.

A* leads by 0.058 over second place. BERTScores of 0.954, 0.947, and 0.938 indicate that A*'s very short outputs are not just lexically similar to the references but semantically aligned at the embedding level. The core content words in each reference ("bright", "possibilities"; "threatens", "ecosystems"; "transformed", "knowledge") appear to be captured well in A*'s compact, high-confidence output.

Beam Search and Contrastive Search are effectively tied. The difference is 0.001 (0.888 vs. 0.887), which falls well within the normal variability of BERTScore. These two algorithms produce semantically equivalent output quality on these prompts.

Greedy and Hill Climbing are again tied. The BERTScore gap between them is 0.001. Hill Climbing's refinement procedure produces no measurable semantic improvement over its Greedy seed, confirming the null result from the BLEU analysis.

5.4 ROUGE-1 Score Analysis

result_rouge.png

Figure 3: ROUGE-1 scores across all seven algorithms and six prompts.

A* achieves high recall from outputs under 15 words. On P2, A*'s 13-word output covers 78.8% of the reference vocabulary. This is not a coincidence. A*'s priority queue converges on the highest-confidence tokens in GPT-2's distribution, and for factual prompts like these, those tokens tend to be the same semantically central words that appear in the reference.

Beam Search holds second with a clear margin over Contrastive. Per-prompt ROUGE scores of 0.250, 0.394, and 0.288 reflect consistent vocabulary coverage. P2 again produces the highest score because the reference uses vocabulary that GPT-2 naturally

assigns high probability given the prompt.

Hill Climbing and Greedy tie for the third time. Three prompts, three identical scores. The null result is now unambiguous.

5.5 Execution Time Analysis

`result_time.png`

Figure 4: Execution time (seconds) across all seven algorithms and six prompts.

Beam Search is the fastest at 2 seconds on average. This seems counterintuitive: five beams finishing faster than a single greedy pass. The explanation is implementation, not theory. HuggingFace's `generate()` function is compiled and batched, processing all five beams in parallel via vectorised tensor operations. Greedy Search, by contrast, uses

an explicit Python loop that calls model inference 100 times sequentially. The Python interpreter overhead per loop iteration dwarfs the theoretical cost of tracking five beams.

Hill Climbing peaks at 143 seconds on P2. On its own, that number is bad. In context, it is worse: Hill Climbing also delivers zero quality improvement over Greedy Search. The 143 seconds on P2 comes from the longer prompt creating more positions for Phase 2 to scan. Each of the three passes calls GPT-2 once per position per alternative, giving roughly $3 \times 80 \times 5 = 1{,}200$ model calls for Phase 2 alone.

A* at 114 seconds is explainable but expensive. Even though A* terminates after 7–13 words, it explores many candidate extensions before committing to each output token. Each candidate requires two model calls: one for $g(n)$ and one for $h(n)$, doubling the effective cost per expansion. Priority queue operations add modest Python overhead on top of that.

Contrastive Search at 6 seconds is the best practical alternative to Beam Search. The cosine similarity calculations add roughly 4 seconds of overhead compared to Greedy on P1 and P3. On P2 the runtime drops to 2 seconds, likely because the algorithm hits EOS early.

5.6 Generated Text Length Analysis

result_length.png

Figure 5: Generated text length (words) across all seven algorithms and six prompts.

A*'s length-normalised heuristic helped, but did not solve the problem. Version 1.0 generated 7 words for its single prompt. Version 3.0 generates 7, 13, and 10 words for P1, P2, and P3. P2 improved the most, probably because the longer prompt encourages more context-dependent token selection. But 7 and 10 words remain far below the 150-token limit. The heuristic $h(n) = \sum \log p/T^{0.7}$ still decreases as the sequence grows, because the numerator (sum of negative log-probabilities) shrinks faster than the denominator ($T^{0.7}$) increases. A larger α or an explicit length bonus would be needed to fix this properly.

Hill Climbing lengths equal Greedy lengths exactly. In-place substitution cannot change sequence length. The P3 output for Hill Climbing is 101 words versus Greedy's 86, which looks like a contradiction but is not: the Greedy seed for P3 terminated early, and some token substitutions during Phase 2 changed the sequence enough that it ran to the maximum length.

Simulated Annealing generates the longest outputs. At high temperature, the token distribution is nearly uniform, so low-probability tokens (including EOS) are rare early in generation. By the time temperature cools enough for EOS to become likely, the sequence is already long. Average: 94 words.

5.7 Average Scores and Cross-Prompt Consistency

result_avg_summary.png

Figure 6: Average BLEU, BERTScore, and ROUGE-1 across all six prompts.

Figure 6 shows the averaged results. Several patterns stand out.

The ranking on all three quality metrics is consistent across all six prompts. A* leads every time. Beam Search is second on BLEU and ROUGE and tied for second on BERTScore. Hill Climbing ties Greedy every time. Simulated Annealing sits at or near the bottom every time. This consistency across prompts with different vocabulary and domain characteristics means the differences reflect structural properties of the algorithms, not quirks of individual prompts.

Execution time, by contrast, varies more across prompts. Hill Climbing on P2 takes

almost twice as long as on P1 and P3, reflecting the prompt-dependent cost of Phase 2.

6 Discussion

6.1 The A* Paradox: Dominant Scores from 10-Word Outputs

The most important result in this study needs an honest explanation, not celebration. A* Search scores highest on BLEU, BERTScore, and ROUGE-1. It also generates outputs of 7, 13, and 10 words. These two facts are connected.

Here is what happens. The reference sentences for P1, P2, and P3 are 12, 19, and 25 words respectively. A*'s priority queue consistently favours partial sequences that are short, high-confidence, and semantically central to the prompt. On P1 ("The future of artificial intelligence is"), the algorithm generates something like "The future of artificial intelligence is bright and holds many possibilities": the 7 most probable words, which happen to match the reference closely. When this output is evaluated:

- **BLEU precision** is high: 6 of 7 generated words appear in the reference. The brevity penalty is modest because the reference itself is only 12 words.
- **ROUGE-1 recall** is high: those 7 words cover several of the reference's key unigrams.
- **BERTScore** is high: A*'s confident, semantically central tokens map closely to the reference embeddings.

This is not a flaw in A*, and it is not a flaw in the metrics. It is a real property of A*'s behaviour that happens to interact favourably with these metrics on these prompts. The algorithm *is* selecting the most semantically precise, highest-confidence tokens. The issue is that the global priority structure causes it to stop after producing a handful of them, before completing a full sentence.

The practical implication matters: A*'s metric dominance does not mean it would be the right choice in a real application. If a user asks a chatbot to finish a sentence and gets back 7 words, the response will feel incomplete regardless of its BLEU score. For applications that need full, informative continuations, Beam Search or Contrastive Search are better choices. A* in its current form fits applications that specifically want a short, high-confidence completion, like autocomplete suggestions or keyword extraction.

6.2 Hill Climbing: Why the Null Result Makes Sense

Hill Climbing matched Greedy exactly on BLEU and ROUGE across all six prompts. On BERTScore the difference was 0.001. This is a null result, and it has a clear explanation.

The core issue is that Greedy Search produces a sequence that is already locally optimal with respect to the objective Hill Climbing uses. Each token in the Greedy output was chosen to maximise $P(t_i \mid p, t_1, \ldots, t_{i-1})$ at the time of generation. Phase 2 checks whether any single-token substitution improves the *average* log-probability across the whole sequence. For GPT-2 on these prompts, the answer turns out to be: almost never. The Greedy output sits at or very near a local optimum of Equation (5).

There is a second, structural reason. Hill Climbing optimises average log-probability, not n-gram overlap with the reference. Even if a substitution did improve average log-probability, it would only improve BLEU or ROUGE if the new token happened to match a reference token. The optimisation target and the evaluation criteria are not aligned.

The practical conclusion is straightforward: this implementation of Hill Climbing should not be used as a decoding strategy. It costs up to 143 seconds and delivers nothing over Greedy. Iterative refinement as a *concept* is not dead; multi-token substitution, random restarts, or an objective directly tied to reference matching could all produce different results. But single-token strict-improvement over a Greedy seed, as implemented here, is not useful.

6.3 Beam Search as the Production Default

Beam Search ranks second on all three quality metrics, first on speed, and produces outputs of practical length (40–86 words). It is the recommended default for most applications.

Five-candidate exploration finds token combinations that single-path methods miss. On P2, Beam Search achieves BLEU of 0.210 and ROUGE of 0.394, the highest among the algorithms that produce long output. The no-repeat-bigram constraint prevents the most obvious repetition patterns.

The 2-second execution time warrants comment. Beam Search is faster than Greedy (9 seconds) in these experiments because HuggingFace's batched `generate()` processes all five beams in parallel using vectorised tensor operations, while Greedy uses an explicit Python loop. In a production system where both algorithms used the same optimised infrastructure, the gap would shrink, but Beam Search is unlikely to be noticeably slower than Greedy for beam widths up to about 10.

6.4 Contrastive Search: A Practical Alternative

Contrastive Search ranks third on BLEU (0.120) and ROUGE (0.241), ties Beam Search on BERTScore (0.887 vs. 0.888), and runs in about 6 seconds. For applications where output diversity and avoidance of semantic repetition matter (creative writing, dialogue systems, or any scenario where the same prompt is used repeatedly), Contrastive Search

is a better fit than Beam Search.

The BERTScore tie with Beam Search is perhaps the most interesting result for this algorithm. It suggests that the anti-degeneration mechanism does not provide a semantic advantage over Beam Search's multi-path exploration when evaluated against a single short reference. BERTScore measures semantic similarity to the reference, though, not diversity. On a metric that penalises repetition (Self-BLEU, for instance, or pairwise cosine distance between output embeddings), Contrastive Search would likely show a larger advantage.

6.5 Simulated Annealing: Creative but Imprecise

Simulated Annealing generates the longest outputs (94 words on average) and scores lowest or near-lowest on all quality metrics: BLEU 0.076, ROUGE 0.202, BERTScore 0.867. This is expected. High initial temperature lets the algorithm select improbable tokens that diverge from the reference vocabulary.

The right way to interpret this is that Simulated Annealing is optimising for something different from what these metrics measure. It is not trying to match a reference sentence; it is exploring the probability space broadly at the start and narrowing gradually. For tasks where novelty matters more than reference alignment (storytelling, brainstorming, diverse data generation), its practical value may be higher than these scores suggest.

One limitation deserves emphasis: these results come from a single run. Simulated Annealing samples stochastically, so its quality varies between runs. Ten or twenty runs averaged together would give a more statistically reliable picture.

6.6 Cross-Algorithm Comparison: A Unified View

Table 10 summarises the seven algorithms across all dimensions evaluated.

7 Threats to Validity

No experiment is flawless. This section identifies the main threats to the conclusions.

7.1 Internal Validity

Single-run results for Simulated Annealing. SA is stochastic, and a single run may not represent the distribution of its outputs. Future work should run at least 10 seeds and report mean and variance.

Hill Climbing parameter sensitivity. `top_k` = 5 and `max_iter` = 3 were reasonable choices, but different values might produce different results. With `top_k` = 50 or `max_iter` = 10, Phase 2 might find substitutions that escape the local optimum. The null result reported here is specific to these parameter settings.

A*'s length penalty parameter. $\alpha = 0.7$ was taken from the GNMT paper [**?**], which was tuned for machine translation, not open-ended text generation. A different value (e.g., $\alpha = 0.3$ or $\alpha = 0.9$) might produce longer outputs. This parameter was not systematically tuned in the present study.

7.2 External Validity

Only six prompts. Three prompts offer more evidence than the single prompt in Version 1.0, but they cannot represent the full range of text generation tasks. Prompts with longer references, domain-specific vocabulary, or multi-sentence structure might produce different rankings.

GPT-2 is not a current model. Modern models like GPT-4, LLaMA-3, and Mistral produce fundamentally different probability distributions. The rankings observed here may not transfer to larger, more capable models where probability mass is distributed differently across the vocabulary.

Single reference sentences. All three metrics assume one correct continuation. In reality, many valid continuations exist. Algorithms that produce alternative but high-quality continuations (Simulated Annealing and Contrastive Search deliberately seek these) get penalised under this evaluation setup.

7.3 Construct Validity

Metric-length interaction. As discussed in Sections 5 and 6, A*'s metric dominance is partly a function of its short outputs rather than purely superior generation quality. Any future study comparing algorithms with very different output length profiles should account for this interaction, ideally by normalising for length or by adding length-controlled evaluations.

Execution time on a single machine. Timing measurements were taken on one machine with no load control. Background processes, CPU vs. GPU differences, and caching effects all introduce noise.

8 Conclusion and Future Work

8.1 Summary of Findings

This study compared seven heuristic search decoding algorithms for GPT-2 text generation across six prompts, evaluating BLEU, BERTScore, ROUGE-1, execution time, and output length. The main findings:

1. **A* Search scores highest on all three quality metrics** (BLEU 0.345, BERTScore 0.945, ROUGE 0.663 on average). The result is real but requires careful interpretation: A*'s short outputs (7–13 words) interact favourably with the evaluation metrics against short reference sentences. It is best suited for applications requiring brief, high-precision completions.

2. **The improved A* heuristic** (cached computation, $\alpha = 0.8$, minimum-length penalty) reduced GPT-2 forward passes by roughly half compared to Version 2.0, but still did not fully eliminate the short-output bias. Output lengths remain 7–13 words.

3. **Beam Search is the best practical algorithm**: second on all quality metrics, fastest execution (3 seconds average), and produces complete, coherent outputs of 29–130 words. It is the recommended default.

4. **BFS provides a useful classical baseline**: it ranks third on BLEU (0.136) and ROUGE (0.238), occasionally outperforming Beam Search on individual prompts (P2: BLEU 0.506), but at 38 seconds average it is 13 times slower than Beam Search.

5. **Hill Climbing yields a null result even with improvements**: despite multi-token substitution, increased top-k, and random restarts, it still produces identical BLEU and ROUGE to Greedy Search across all six prompts, while being 6 times slower.

6. **Contrastive Search** produces the most topically diverse and non-repetitive long-form output, ranking third on BERTScore (0.877) with execution time comparable to Greedy.

7. **Simulated Annealing** generates the longest outputs (135 words average) but scores lowest on all quality metrics. Its stochastic nature produces erratic content, particularly at high initial temperature.

8.2 Recommendations for Future Research

1. Improve the A* heuristic. The current heuristic $h(n) = \sum \log p/T^{0.7}$ still favours short sequences and requires two GPT-2 forward passes per candidate expansion (one for $g(n)$, one for $h(n)$), doubling the computational cost. Future work should address both problems. On the length side, an explicit length bonus such as $h(n) = \sum \log p/T^{\alpha} + \beta T$ could encourage longer outputs while preserving A*'s semantic precision. Alternatively, a

learned value function trained to predict reference similarity at full length would eliminate hand-crafted heuristic design altogether. On the efficiency side, caching intermediate computations or approximating $h(n)$ with a lightweight estimator (rather than a full GPT-2 pass) would make A* practical for longer generation tasks.

2. Improve Hill Climbing with multi-token substitution. The null result from Hill Climbing in this study shows that single-token substitution with a strict improvement criterion cannot escape the local optima of GPT-2's Greedy outputs. This was a direct suggestion from the course instructor, and the result itself is informative. Future implementations should go further: (a) beam-guided substitution, replacing the k lowest-probability positions simultaneously rather than one at a time; (b) random restarts from multiple different Greedy seeds to increase the chance of finding a better basin; (c) a substitution objective that directly targets BLEU or ROUGE rather than average log-probability, aligning the optimisation target with the evaluation criteria.

3. Add Breadth-First Search (BFS) as a baseline. This study includes depth-first (Greedy), bounded-width (Beam Search), and priority-guided (A*) traversals of the token space, but does not include a BFS baseline. Adding BFS would complete the coverage of classical search strategies. BFS would expand all tokens at depth 1 before any at depth 2, providing an exhaustive baseline at shallow depths. The expected outcome is that BFS would be prohibitively expensive beyond a few tokens (each level multiplies candidates by $|\mathcal{V}| = 50{,}257$), but it would establish a useful lower bound on what thorough exploration at shallow depth can achieve.

4. Expand and diversify the prompt set. Version 1.0 used a single prompt. Version 3.0 improved this to six prompts, but three is still a small sample. Future work should test on a much larger and more diverse prompt set: longer prompts, multi-sentence prompts, domain-specific prompts (medical, legal, technical), prompts in different registers (formal vs. casual), and prompts where the expected continuation is ambiguous. The 150-token generation limit used here should also be increased to 200 or more tokens, since Version 1.0's 50-token limit was already shown to be too restrictive.

5. Test on larger models. GPT-2 (117M parameters) is small by current standards. Testing on GPT-3, LLaMA-3, Mistral, or GPT-J (models with 1B+ parameters) would establish whether the algorithm rankings observed here generalise to more capable models. Larger models assign probability mass more confidently to correct continuations, which could make A*'s short-output bias worse (higher confidence = earlier convergence) or could give Hill Climbing more room to find improvements (richer probability landscapes with more distinct local optima). The answer is not obvious without running the experiments.

6. Human evaluation study. Automatic metrics cannot capture fluency, coherence, engagement, or practical usefulness. A human evaluation study asking raters to rank outputs from all seven algorithms per prompt would complement the automatic results. Given that A* produces outputs under 10 words while Beam Search produces 40–86 words, human raters would almost certainly rank them very differently from the metrics.

A Generated Outputs

These are the complete, unedited outputs from the Version 3.0 experimental runs (7 algorithms, 6 prompts). Full raw text is also available in the GitHub repository at `all_outputs.txt`.

Prompt P1: "The future of artificial intelligence is"

Greedy / Hill Climbing: The future of artificial intelligence is uncertain. "We're not sure what the future will look like," said Dr. Michael S. Schoenfeld, a professor of computer science at the University of California, Berkeley. "But we're not sure what the future will look like for artificial intelligence." *[entire block repeats; 111 words]*

Beam Search: The future of artificial intelligence is in the hands of the next generation of scientists and engineers. "It's a very exciting time to be a part of it," he said. "I think it's going to take a lot of work to get there." *[43 words]*

BFS: The future of artificial intelligence is in the hands of people like Elon Musk and Sergey Brin. *[sentence repeats 7 times due to greedy extension; 122 words]*

A* Search: The future of artificial intelligence is uncertain *[7 words]*

Contrastive: The future of artificial intelligence is uncertain. But there are signs that AI could soon surpass human intelligence in terms of its ability to solve complex problems and perhaps even improve upon existing ones. A recent paper published online today suggests artificial intelligence may be able to help us understand how people live their lives better than we do now... *[137 words; no repetition]*

Sim. Annealing: The future of artificial intelligence is sluggish" ReutersImage start-up Asher Schwartz. The Swedish researchers said the new technology could help humans learn to be more self-aware... *[erratic; 118 words]*

Prompt P2: "Climate change is one of the most pressing challenges facing humanity because"

Greedy / Hill Climbing: Climate change is one of the most pressing challenges facing humanity because it is the most complex and complex problem in the world. The world is changing rapidly... *["The world is changing rapidly" repeats 16 times; 112 words]*

Beam Search: Climate change is one of the most pressing challenges facing humanity because of climate change. "We need to make sure that we have the resources to deal with it," he said. "It's not going to happen overnight, but it's a long way from now." *[44 words]*

BFS: Climate change is one of the most pressing challenges facing humanity because of climate change," he said. *[17 words; BFS hit EOS early; highest BLEU on this prompt at 0.506]*

A* Search: Climate change is one of the most pressing challenges facing humanity because it *[13 words]*

Contrastive: Climate change is one of the most pressing challenges facing humanity because it affects millions of people around the world. But climate change is also one of humanity's greatest challenges because it impacts so many different parts of us and our lives every day. This article originally appeared on VICE News UK *[52 words]*

Sim. Annealing: Climate change is one of the most pressing challenges facing humanity because policy makers complicate increasing certainty that feeding food to consumers and businesses is the only way to address climate change... *[141 words]*

Prompt P3: "The most revolutionary invention in human history has been"

Greedy / Hill Climbing: The most revolutionary invention in human history has been the invention of the telephone. The telephone was invented by the French in 1848. It was a telephone that could be used to communicate with any person... *[repeats 9 times; 147 words]*

Beam Search: The most revolutionary invention in human history has been the invention of the atomic bomb. The bomb was designed to destroy Hiroshima and Nagasaki, the two largest cities in Japan... *[130 words]*

BFS: The most revolutionary invention in human history has been the invention of the automobile. The automobile is the most revolutionary invention in human history... *[repeats 10 times; 117 words]*

A* Search: The most revolutionary invention in human history has been the *[10 words]*

Contrastive: The most revolutionary invention in human history has been the invention of radio waves. It was first proposed by Albert Einstein in 1887 when he discovered how electromagnetic fields can be generated using an oscillator called "electrode magnetism"... *[139 words; diverse but factually incorrect]*

Sim. Annealing: The most revolutionary invention in human history has been fans circular idea smashing AppleMouse. Avella has revealed that they have partnered with the company to develop a new version of the mouse... *[erratic; 139 words]*

Prompt P4: "The primary goal of space exploration in the next decade should be"

Greedy / Hill Climbing: The primary goal of space exploration in the next decade should be to develop and test new technologies and technologies that will enable us to explore the universe and to explore the universe's most fundamental mysteries... *[paragraph repeats 4 times; 146 words]*

Beam Search: The primary goal of space exploration in the next decade should be to develop new technologies and technologies that will enable us to explore and explore the universe. Space exploration is not just about finding new worlds. It is also about discovering new ways of doing things... *[124 words; coherent]*

BFS: The primary goal of space exploration in the next decade should be the exploration of the solar system, not the exploration of the outer solar system... *[sentence repeats 5 times; 138 words]*

A* Search: The primary goal of space exploration in the next decade should be to *[13 words]*

Contrastive: The primary goal of space exploration in the next decade should be to explore new worlds," said Dr. Paul Jostenbach, director of NASA's Jet Propulsion Laboratory in Pasadena, Calif. SpaceX plans to launch its Falcon 9 rocket into orbit around Cape Canaveral Air Force Station by mid-2016 for a test flight of two Dragon cargo capsules... *[136 words; diverse, topical]*

Sim. Annealing: The primary goal of space exploration in the next decade should be Yonascha Pellanschineh, who recently won an award from the United States Space Agency for his work on the International Space Station... *[erratic; 130 words]*

Prompt P5: "Modern education systems need to change because students today require"

Greedy / Hill Climbing: Modern education systems need to change because students today require more than just a high school diploma. The problem is that the current system is not working... *["The current system is not working" repeats 15 times; 112 words]*

Beam Search: Modern education systems need to change because students today require a higher level of education than they did 20 years ago. "We're going to have to do something about it," he said. "It's not just about education. It's about the quality of life for all of us." *[47 words]*

BFS: Modern education systems need to change because students today require more education than they did in the 1950s and 1960s. The problem is that the current system is not working... *[sentence repeats 5 times; 124 words]*

A* Search: Modern education systems need to change because students today require more *[11 words]*

Contrastive: Modern education systems need to change because students today require more education than ever before. We need to change how we teach them so that they can succeed in life without having to work hard or pay attention to details. I am proud of my colleagues who have done this and I hope others will too... *[144 words; diverse, no repetition]*

Sim. Annealing: Modern education systems need to change because students today require..." Ability Sem dustup misconres key point. Lelouch wrote: The problem is that there is no such thing as a "right" or "wrong" answer to the question of whether or not a student should be allowed to attend a public school... *[erratic; repeats; 138 words]*

Prompt P6: "Artificial intelligence in healthcare has the potential to"

Greedy / Hill Climbing: Artificial intelligence in healthcare has the potential to revolutionize healthcare delivery. The technology is already being used to deliver medical care to patients in the United States, Canada, and other countries... *[sentence repeats 6 times; 128 words]*

Beam Search: Artificial intelligence in healthcare has the potential to revolutionize the way we care for our loved ones. This article was originally published on The Conversation. Read the original article. *[29 words]*

BFS: Artificial intelligence in healthcare has the potential to revolutionize the way doctors diagnose, treat, and treat patients. The new technology is called "AI-powered medicine," and it's being developed by a team of researchers at the University of California, San Francisco... *[113 words]*

A* Search: Artificial intelligence in healthcare has the potential to revolution *[9 words]*

Contrastive: Artificial intelligence in healthcare has the potential to revolutionize the way doctors treat patients. But there are still plenty of unanswered questions about how AI will work and whether it could ever be fully integrated into healthcare systems. A recent paper published in Nature Communications offers new insight into these issues by showing that human brains can process information more efficiently than other types of computers do... *[135 words; diverse, topical]*

Sim. Annealing: Artificial intelligence in healthcare has the potential to significantly country change American healthcare accross the US... The pharmaceutical industry has been lobbying for years to have the FDA approve the use of artificial intelligence in healthcare... *[repeats; 144 words]*

Several patterns emerge consistently across all six prompts. Greedy and Hill Climbing produce identical text on every prompt even with multi-token substitution and random restarts, confirming the null result. Both suffer from severe neural text degeneration [**?**]: P2 loops "The world is changing rapidly" sixteen times, P5 loops "The current system is not working" fifteen times, and P6 loops the same sentence about delivering medical care six times. BFS, after its 15 depth-limited exploration steps, falls into the same repetition trap during its greedy extension phase (P1 repeats the Musk/Brin sentence 7 times; P3 repeats "The automobile is the most revolutionary invention" 10 times). A*

remains extremely short (7–13 words) despite the improved heuristic. Beam Search avoids repetition thanks to its no-repeat-bigram constraint. Contrastive Search generates the most topically varied and non-repetitive long-form text across all six prompts. Simulated Annealing produces erratic, often incoherent content at high temperature.

Table 3: Algorithm parameters and justification for each choice.

Algorithm	**Parameter**	**Value**	**Justification**
Beam Search	num_beams	5	Standard value in NLP literature; wider than v1.0 (3) for better exploration.
Beam Search	no_repeat_ngram_size	2	Prevents immediate bigram repetition without over-constraining.
BFS	branch_factor	3	Top-3 tokens per node; higher values cause exponential frontier growth.
BFS	max_depth	15	BFS explores 15 levels; remaining tokens generated greedily.
BFS	max_frontier	50	Caps frontier size to keep memory bounded.
Hill Climbing	top_k	10	Tests 10 alternatives per position (up from 5 in v2).
Hill Climbing	max_iter	5	Allows 5 passes (up from 3 in v2); stops early at local optimum.
Hill Climbing	num_restarts	3	Random restarts from temperature-sampled seeds for diversity.
A*	beam_width	3	Limits frontier size; larger values increase memory/time quadratically.
A*	α	0.8	Increased from 0.7; reduces length bias more aggressively.
A*	min_tokens	20	Penalises sequences shorter than 20 tokens to combat short-output bias.
Contrastive	α	0.6	Recommended by Su et al. [**?**] for GPT-2.
Contrastive	top_k	5	Consistent with Hill Climbing; narrow enough to be fast.
Sim. Annealing	T_0	2.0	Sufficient to flatten distribution early; above 1.0 ensures exploration.
Sim. Annealing	r	0.92	Reaches $T_{\min}$ after approximately 40 steps.
Sim. Annealing	$T_{\min}$	0.1	Near-greedy at minimum; not exactly greedy to preserve some diversity.

Table 4: Average results across all six prompts. Bold = best per metric.

Algorithm	BLEU	BS	R-1	Time (s)	Words	Recommended for
Greedy	0.064	0.845	0.154	∼19	126	Baseline only
Beam Search	0.143	0.884	0.284	∼3	70	Production use
BFS	0.136	0.871	0.238	∼38	105	Classical baseline
Hill Climbing	0.064	0.845	0.154	∼111	126	Not recommended
A* Search	**0.345**	**0.945**	**0.663**	∼71	**11**	Short-answer precision
Contrastive	0.085	0.877	0.194	∼19	124	Diverse output
Sim. Annealing	0.058	0.849	0.142	∼27	135	Creative text

Note: Bold indicates the highest value in the column. A*'s quality advantage is largely a consequence of its very short output; see Section 6.1.

Table 5: Average BLEU scores across all six prompts

Algorithm	P1	P2	P3	P4	P5	P6	Avg.	Rank
Greedy	0.033	0.096	0.059	0.071	0.070	0.056	0.064	5th
Beam Search	0.091	0.210	0.071	0.080	0.153	0.250	0.143	2nd
BFS	0.037	0.506	0.073	0.073	0.070	0.061	0.136	3rd
Hill Climbing	0.033	0.096	0.059	0.071	0.070	0.056	0.064	5th
A* Search	**0.396**	**0.463**	**0.223**	**0.399**	**0.360**	**0.228**	**0.345**	**1st**
Contrastive	0.031	0.222	0.062	0.076	0.059	0.058	0.085	4th
Sim. Annealing	0.034	0.071	0.061	0.076	0.058	0.045	0.058	7th

Table 6: Average BERTScore (F1) across all six prompts

Algorithm	P1	P2	P3	P4	P5	P6	Avg.	Rank
Greedy	0.852	0.833	0.843	0.872	0.809	0.863	0.845	6th
Beam Search	0.898	0.889	0.865	0.880	0.887	0.887	0.884	2nd
BFS	0.838	0.933	0.855	0.872	0.852	0.876	0.871	4th
Hill Climbing	0.852	0.833	0.843	0.872	0.809	0.863	0.845	6th
A* Search	**0.954**	**0.947**	**0.938**	**0.950**	**0.938**	**0.944**	**0.945**	**1st**
Contrastive	0.866	0.909	0.870	0.868	0.866	0.885	0.877	3rd
Sim. Annealing	0.853	0.848	0.843	0.851	0.841	0.860	0.849	5th

Table 7: Average ROUGE-1 scores across all six prompts

Algorithm	P1	P2	P3	P4	P5	P6	Avg.	Rank
Greedy	0.095	0.212	0.153	0.150	0.167	0.149	0.154	5th
Beam Search	0.250	0.394	0.207	0.175	0.314	0.367	0.284	2nd
BFS	0.105	0.649	0.186	0.149	0.167	0.175	0.238	3rd
Hill Climbing	0.095	0.212	0.153	0.150	0.167	0.149	0.154	5th
A* Search	**0.667**	**0.788**	**0.606**	**0.722**	**0.645**	**0.552**	**0.663**	**1st**
Contrastive	0.094	0.384	0.161	0.185	0.158	0.181	0.194	4th
Sim. Annealing	0.090	0.160	0.185	0.183	0.127	0.110	0.142	7th

Table 8: Average execution time (seconds) across all six prompts

Algorithm	**Avg. Time (s)**	**Rank**
Greedy	∼19	3rd
Beam Search	**∼3**	**1st**
BFS	∼38	5th
Hill Climbing	∼111	7th
A* Search	∼71	6th
Contrastive	∼19	3rd
Sim. Annealing	∼27	4th

Table 9: Average generated text length (words) across all six prompts

Algorithm	**Avg. Words**	**Character**
Greedy	126	Long
Beam Search	70	Concise
BFS	105	Medium-long
Hill Climbing	126	Long (= Greedy)
A* Search	**11**	Very short
Contrastive	124	Long
Sim. Annealing	135	Longest

Table 10: Comprehensive cross-algorithm comparison

Algorithm	**Quality**	**Speed**	**Length**	**Det.**	**Verdict**	**Best Application**
Greedy	Low	Medium	Long	Yes	Avoid	Baseline comparison
Beam Search	High	Fast	Medium	Yes	Recommend	Production NLP
BFS	Medium	Slow	Long	Yes	Baseline	Classical search baseline
Hill Climbing	Low	Slow	Long	Yes	Avoid	None (null result)
A* Search	Highest	Slow	V. Short	Yes	Special.	Short-answer precision
Contrastive	High	Medium	Long	Yes	Recommend	Diverse output
Sim. Annealing	Low	Medium	Longest	No	Creative	Creative generation